claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-a51aeebbb6a26849d.jsonl`
- SHA-256: `9757d7694ccf487c9600ff10a5db4d66a4ba5220aae09009681b99320ab51dd1`
- Source modified: `2026-07-08T06:39:06+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:36:06.694Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.
A prior reviewer raised this finding on the DESIGN-DOC ACCURACY (docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md) dimension:
  SEVERITY: low
  FILE: docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:68
  SUMMARY: The ~30s 'Accept?validate' gap is attributed (as [verified]) to a CP scratch re-download + re-hash, but the cited runlog labels that same phase differently.
  CLAIMED FAILURE: §4.1 (line 68) / §1 finding (b) / §0 say the ~30s = 'CP re-download to scratch (acquire_local_input + re-hash)', tagged [verified] against runlog §9. But runlog line 118 labels the ~30s 'Accept ? validation start' phase 'stage upload to serving bucket'. The two sources give different causes for the same measured number, and the doc calls it 'unlogged' while the runlog does label it. (Code actually backs the doc: uploads.py streams straight into `<input_root>/uploads/?` in the serving bucket with no bucket-to-bucket stage-copy ? so the runlog label is the imprecise one ? but the [verified] tag still conflicts with its own cited evidence.)
  PROPOSED FIX: Reconcile the two: either correct the runlog phase label, or retag §4.1's causal claim [researched]/[design] and note the runlog labels it as a stage-copy. Optionally clarify that win #2 removes only the (fast) re-hash and it is P2b ? not this PR ? that removes the download bulk of the 30s.

Adversarially VERIFY it against the real code/doc in C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only

if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. user (2026-07-08T06:36:21.300Z)

```
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the
next time Git touches it
 backend/api/job_worker.py                          |   5 +
 backend/config/models.py                           |  10 +
 backend/orchestration.py                           |  51 ++++-
 .../L4_PRIMARY_TOPOLOGY.md                         | 225 ++++++++++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   2 +
 tests/test_config_deployment.py                    |   2 +
 tests/test_video_id_trust.py                       | 173 +++++++++++++++
 7 files changed, 466 insertions(+), 2 deletions(-)
```

3. user (2026-07-08T06:36:22.520Z)

```
1       # L4-primary pipeline topology ? collapse the pipeline onto the worker, shrink the
control-plane to a thin router
2
3       > **Status:** `DRAFT` ? **owner-review** (recommends *against* the epic's proposed
direction #1; see §0/§8) · **Owner:** Avi · **Created:** 2026-07-08 · **Last updated:**
2026-07-08
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Issue:** [#524](https://github.com/Khelsutra/badminton-highlight-indexer/issues/524)
(epic) · **Builds on:** [#521](https://github.com/Khelsutra/badminton-highlight-
indexer/issues/521) (`phase_placement`) · **Motivated by:**
[#520](https://github.com/Khelsutra/badminton-highlight-indexer/issues/520) (the tracing that
surfaced the data-movement finding)
6       > **Relation to existing docs:** extends
[COMPUTE_DECOUPLED_SERVING/README.md](./README.md) (ADR-014) and [`PHASE_PLACEMENT.md`
(#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md); it does **not** supersede them ? it
answers the *topology* question they left open and sequences the migration on top of
`phase_placement`.
7       > **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog),
`[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]`
(proposed). Not legal/financial advice.
8
9       ---
10
11      ## 0. TL;DR
12
13      The 2026-07-08 live CUJ moved one 1.83 GiB clip **3×** (upload?bucket, bucket?control-
plane to validate, bucket?worker to detect) and pinned the CPU-only 2-vCPU control-plane for
**~16 min** of heavy work (validation 183 s + stitch ~12m55s). The epic proposes fixing this by
**uploading directly onto the L4** and async-copying to the bucket. **This design recommends
*against* that direction** and quantifies why: a same-region bucket?compute transfer inside GCP
is **LAN-fast and not billed as internet egress** `[researched]`, so the round-trip that upload-
onto-L4 saves is worth **~seconds and ~$0**, while keeping an L4 up to *receive* a **minutes-
long** upload burns **~175 s of idle GPU per run (~$0.08?0.16)** `[design]` and *opens* a
durability exposure window instead of closing one.
14
15      The cheaper, more durable, already-ratified path is the opposite: **keep the upload
streaming to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever
starts), and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ?
which is exactly what **#521's `phase_placement` already wires as a config flip**. The residual
"~30 s pre-validation gap" is then killed not by moving the *upload* but by the control-plane
**never downloading the file at all** (it delegates validation to the L4 per #521, and trusts
the already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops
from 3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with
```

zero idle-GPU cost and **zero exposure window**.

Actionable now (this PR): the design doc + **win #2** (drop the redundant control-plane re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ? this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is **rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.

1. Problem & goal

Why now. Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9 `[verified]`) produced two findings:

1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to `gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix, `backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane re-downloads the whole clip to scratch to validate ? `_execute_processing` ? `storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect (`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch (~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane, serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates past ~5 concurrent users.

The concrete outcome we want. The L4 worker becomes the primary compute for the whole pipeline; the control-plane shrinks to a **thin router** (auth · dispatch · DB · SPA + job status · signed-URL download) that does **no heavy decode/encode** and, ideally, **never pulls the full file**. Every move must be **reversible by config** with a **bucket-mediated fallback**, and land as small PRs on top of `origin/master`.

The hard question this doc must resolve (compute topology). "Upload directly to the L4" needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch, no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a hybrid** presign/redirect. §4 answers it with numbers.

Read to get current: [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-highlight-indexer/blob/feat/521-compile-on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md) ? the `phase_placement` knob this sequences on, [README.md](./README.md) §2 decisions **D4/D7/D16** (the ratified constraints that bound the options), and the runlog §9 (the evidence).

2. Decisions locked

#	Decision	Source / date	Implication
L1	**Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed for durability.	this doc `[design]` + #480 `[verified]`	No exposure window; upload cost stays on the cheap I/O path.
L2	**Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via `phase_placement` (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)** (Jobs, not Services).	#521 `[verified]` + D7/D16	The "L4-primary" vision is realized by **config flips**, not a rearchitecture.
L3	**Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP trusts the submit-time md5 ? no re-hash).	this doc `[design]`	The CP stops pulling the full file when validation is delegated; movement ? 3×?2×.
L4	**Reject persistent-GPU (GCE VM / Service-with-GPU) at current volume.** Breakeven vs scale-to-zero is a **~51 % on-demand / ~31 % spot duty cycle** (? 2,600?4,300 runs/mo sustained) `[design]`; we are at pilot volume. Revisit only behind a **new ADR** at sustained load.	§4.2 cost model `[design]`	Preserves per-run billing (D7/D8); avoids ~$382?637/mo idle.
L5	**Reject "upload onto the L4."** It saves a ~free, seconds-fast intra-region LAN

pull at the cost of ~175 s idle GPU/run **and** opens a durability window. | §4.3 `[design]` |
The epic's proposed direction #1 is not adopted; owner sign-off requested (§8). |
41 | L6 | **Win #2 (drop the re-hash) is config-guarded + reversible**
(`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`**
(the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-
regression; one-flag rollback. |
42
43   *(IDs are `L#` to avoid colliding with README.md's `D#`. These refine ? never override ?
D4/D7/D16.)*
44
45   ## 3. Foundation ? the ratified constraints this must live inside
46
47   The topology is **not a blank slate** ? ADR-014 (README.md §2) already ratified the
serving substrate. The options in §4 are scored **against** these, and any option that breaks
one needs a new ADR, not just this doc:
48
49 | Ratified decision | What it pins | Consequence for #524 |
50 |---|---|---|
51 | **D4** | In cloud-serving, **stitch runs on Cloud Run** (co-located, metered). | The
live system's CP-stitch is a *deviation* from D4; #521 *realizes* D4. #524 is mostly
completing an already-ratified decision, not inventing one. |
52 | **D7** | **Scale-to-zero, no warm pool** (a ~$500/mo idle GPU "would break per-video
billing"). | Any persistent-GPU option (B, Service-with-min-instances) **contradicts D7** ? new-
ADR territory (§4.2). |
53 | **D16(a)** | **Cloud Run Jobs** (not Services-with-GPU), **bucket-poll** for
completion. | Option A (Service-with-GPU inbound upload) **contradicts D16(a)**. The Job model
is load-bearing (reuses `JobWaiting`/`claim_due_job`). |
54 | **D16(b)** | Stitch encode = libx264 for parity; **NVENC deferred behind a knob**. |
#521 lands exactly that knob (`cloud_stitch_encoder`), using this CUJ as the cost data D16(b)
said to revisit on. |
55 | **D16(c)** | Input cap hard-reject `< 2 GB`. | The 1.83 GiB CUJ clip is *at* the cap;
the topology must stay sane at 2 GB (upload minutes, pull seconds). |
56
57   **Key reframing this foundation gives us:** the owner's "reliable persistent disk"
requirement is *already met* by the **regional GCS bucket** ? a durable, co-located, cheap
"disk" the upload already writes to directly. The persistent-VM instinct solves a problem
(durable local disk) that the bucket-first design doesn't have.
58
59   ## 4. Design / architecture ? resolving the topology question
60
61   ### 4.1 The measured pipeline (evidence base) `[verified]`
62
63   From runlog §9 (rev `rally-control-plane-00019`, "First CUJ.MP4", 1.83 GiB / 174.17 s /
4K-class):
64
65 | Phase | Wall | Machine | GPU needed? | Notes |
66 |---|---|---|---|---|
67 | Upload (client ? bucket) | **~2m55s** | client uplink | no | 1.83 GiB @ ~84 Mbps ?
owner bandwidth, not infra |
68 | Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to
scratch (`acquire_local_input` + re-hash) |
69 | Validation (#512 one-pass) | **183 s** | control-plane 2 vCPU | no (CPU-bound) |
scene_cut/blur/relevance shared decode |
70 | Dispatch ? worker ready | ~55 s | Cloud Run Job | ? | image pull + GCSFuse +
torch/cuda init (cold start) |
71 | Detection (WASB) | **161.6 s** | L4 (cuda) | **yes** | the only genuinely GPU-bound
phase |
72 | Ingest + finalize | ~4 s | control-plane | no | intervals ? DB |
73 | Compile / stitch | **~12m55s** | control-plane 2 vCPU | no (NVENC would need GPU) |
11× 4K re-encode+watermark |
74
75   **Two facts that drive the whole design:**
76   - **Only *detection* is GPU-bound.** Validation is CPU/decode-bound; stitch is encode-
bound (NVENC *wants* a GPU but libx264 doesn't *need* one). So "collapse onto the L4" is about
**co-locating CPU work next to the GPU on a box that's already up and already holding the

bytes**, not about the GPU per se.
77   - **Upload dwarfs compute** (175 s upload vs 161.6 s GPU-detect). Anything that keeps a
GPU allocated *during upload* roughly **doubles** the GPU-allocated time for zero compute.
78
79   ### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08)
`[verified]` rates / `[design]` per-run
80
81   Rates pulled live from the Catalog API (`cloudbilling.googleapis.com`, project `gen-
lang-client-0306026826`):
82
83 | Resource | Rate | Source |
84 |---|---|---|
85 | Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog
`[verified]` |
86 | Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87 | Cloud Run memory | $0.0000024 / GiB·s | Catalog `[verified]` |
88 | ? **L4 Job @ 8 vCPU / 32 GiB (running)** | **? $1.705 / hr** ($0.000474/s) | derived
`[design]` |
89 | GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **~$637 /
mo** | Catalog `[verified]` |
90 | GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog
`[verified]` |
91 | Cloud Run **GPU Service, warm** (min-instances=1, 4 vCPU/16 GiB) | ? $1.256 / hr =
~$917 / mo idle | derived `[design]` |
92 | Balanced PD | $0.11 / GiB·mo | Catalog `[verified]` |
93
94   **Per-run GPU-allocated cost (the term that varies by topology):**
95
96 | Topology | GPU-allocated time / run | GPU cost / run | ? vs today | Concurrency wall?
|
97 |---|---|---|---|---|
98 | **Today** (detect-only on L4; validate+stitch on CP) | ~222 s (cold+detect+io) |
~$0.105 | ? | **Yes** ? ~16 min CP CPU/run |
99 | **#521 + win #2** (validate+detect+stitch on L4; upload?bucket) | ~312 s (validate 60
+ detect 162 + stitch 60 + io 30) | **~$0.148** | **+$0.04** | **No** ? CP is thin |
100 | **Upload-onto-L4** (GPU up during upload too) | ~487 s (+175 s idle upload) |
~$0.231 | **+$0.13** | No, but idle-GPU tax |
101 | **Persistent GCE VM** (on-demand) | billed 24/7 regardless | **$637/mo flat** | breaks
per-run billing | No |
102
103   **Breakeven (persistent vs scale-to-zero):** a persistent on-demand g2 ($0.872/hr) beats
the scale-to-zero Job ($1.705/hr *while running*) only at a **duty cycle ? $0.872/$1.705 ? 51
%** (spot: ? 31 %). At ~312 s/run that is **~2,600?4,300 runs/month sustained, 24/7**
`[design]`. Pilot volume is *handfuls/day*. **Scale-to-zero wins by ~2 orders of magnitude** and
preserves per-run billing (D7/D8). A persistent GPU is also **idle >95 % of the time** at our
volume ? the ~$382?637/mo would be almost pure waste.
104
105   ### 4.3 The three topology options, scored
106
107   **Option A ? Cloud Run Service-with-GPU hosting upload + pipeline.**
108   - *Latency:* scale-to-zero ? same ~55 s cold start as the Job; **min-instances=1**
removes cold start but pins a **~$917/mo warm idle GPU**.
109   - *Cost:* to accept the upload, the GPU Service instance must be **up for the whole 175
s upload** ? the idle-GPU tax of §4.2 on *every* request, plus the warm-idle bill if min-
instances>0.
110   - *Ops/parity:* **contradicts D16(a)** (Jobs, not Services) and **D7** (scale-to-zero) ?
abandons the `JobWaiting`/`claim_due_job`/bucket-poll machinery that's shipped and tested. A
Service also needs request-timeout/concurrency tuning the batch Job sidesteps.
111   - **Verdict: reject.** Buys an inbound endpoint we don't need (the bucket is the inbound
endpoint) at the cost of the idle-GPU tax + two ratified-decision reversals.
112
113   **Option B ? GCE L4 VM (persistent, reliable PD).**
114   - *Reliability:* real, attached, reliable PD ? the owner's stated appeal. But **the
bucket already gives us durable co-located storage** (L1), so the PD solves a non-problem.
115   - *Cost:* **$382?637/mo flat**, >95 % idle at pilot volume; breaks per-run billing

(D7/D8). Spot is cheaper but **preemptible** ? directly against the "reliable" motivation.
116   - *Ops:* we now **own a VM** ? patching, disk-fill, crash-recovery, autoscaling-group if we ever need >1. The Job model has none of this.
117   - **Verdict: reject at current volume; owner-gated future ADR** if sustained load crosses the §4.2 breakeven.
118
119   **Option C ? Hybrid: control-plane presigns/redirects the upload to a just-spun-up L4, pipeline runs there, async copy to bucket.**
120   - *The fatal timing problem:* compute needs the **whole** file, so the L4 cannot start useful work until the upload finishes. Spinning the L4 up *to receive* the upload means the GPU is **allocated and idle for the entire 175 s upload** ? the §4.2 idle tax, same as A. Upload and compute **cannot overlap**.
121   - *Durability:* introduces an **exposure window** (upload-complete ? async-mirror-confirmed) where the only copy is on L4 local disk; a preemption/crash in that window **loses the source** unless the client re-uploads (see §4.5).
122   - **Verdict: reject.** It pays the idle-GPU tax *and* takes on a durability window, to save a transfer that is ~free and seconds-fast.
123
124   **Option A? (recommended) ? co-located Jobs, bucket-mediated upload (the L1?L3 design).**
125   - Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase_placement` (#521). Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.
126   - *Data movement:* **1 client upload + 1 intra-region LAN pull** ? the floor. (Detection and validation/stitch all run in **one** Job execution, so it's one pull, not two.)
127   - *Cost:* +$0.04/run GPU vs today, **?~16 min CP CPU/run** (the actual scaling win). No idle tax, no warm bill, no VM.
128   - *Durability:* **no exposure window** ? the bucket copy exists before the Job starts.
129   - *Ops/parity:* **honors D4/D7/D16**; reuses the shipped dispatch/poll path; every phase is a **config flip with an in-process fallback** (#521's `_dispatch_*_remote` already logs-and-falls-back to in-process when the box isn't cloud-wired).
130
131   > **The one-line intuition:** the bucket *is* the co-located reliable disk; the GPU should attach only for the ~5 min of compute, pulling from it at LAN speed ? never sit idle for the ~3 min upload.
132
133   ### 4.4 The minimal control-plane surface (the "thin router")
134
135   After A?, the control-plane does **only** I/O-light, decode-free work ? trivially horizontally scalable (the one caveat is SQLite; a Postgres move is out of scope, noted in §6):
136
137 | Keep on the control-plane | Why it must stay | Code anchor `[verified]` |
138 |---|---|---|
139 | **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |
140 | **Upload endpoint** (streams parts ? bucket) | Bounded-memory, no decode (#480); the durable-copy producer | `backend/api/uploads.py` |
141 | **Dispatch / routing** (submit job, dispatch Cloud Run Job, bucket-poll) | Owns the async job lifecycle (`JobWaiting`/`claim_due_job`) | `backend/orchestration.py`, `backend/compute/cloud_run.py` |
142 | **DB writes** (jobs, videos, `play_segments`, validation verdict) | The control-plane owns the SQLite DB (worker uses a throwaway scratch DB) | `backend/infrastructure/database*.py` |
143 | **SPA serving + job status** (`/api/jobs/{id}` polling) | The UX surface | `backend/api/jobs.py`, `backend/ui/` |
144 | **Signed-URL download** (`/api/highlights` 307) | Reel is already in the bucket; CP just signs | `backend/main.py::get_highlights` |
145 | **~~Validation~~** | ? **L4** via `phase_placement.validation` (#521) | delegated |
146 | **~~Stitch/compile~~** | ? **L4** via `phase_placement.compile` (#521) | delegated |
147 | **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |
148
149   ### 4.5 The async copy-to-bucket durability model (spec ? needed *only* if a future upload-onto-L4 path is ever adopted)

The recommended A? design **has no async-copy step** (the bucket copy is the *upload*, produced before compute). This section specs the model the epic asked for, so that if the owner overrides §8 and adopts upload-onto-L4, the durability semantics are pinned rather than improvised:

- **When the mirror runs:** kick a background `storage.put(local ? gs://?/uploads/?)` **immediately on upload-complete** (not "during segmentation" ? starting it later only lengthens the exposure window; there's no compute reason to delay it). Run it concurrently with validation/detect.
- **Failure/resume:** retry with capped exponential backoff; the mirror is **idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the source is still on the client). **Never** report success until the mirror is **confirmed** (object exists + size/md5 match).
- **Exposure window:** upload-complete ? mirror-confirmed, the **only** durable copy is L4 local PD. Window length ? mirror wall (1.83 GiB intra-region ? seconds?tens of seconds `[design]`). A preemption in-window = re-upload. **This window is the entire reason A? is preferred** ? A?'s window is *zero*.
- **Ack contract:** the client keeps the source until the API returns `mirror_confirmed:true`; only then may it release it. (A? needs no such contract ? bucket-first.)

5. Phased migration ? every step a config flip with a bucket-mediated fallback

Built on #521's `resolve_phase_placement(cfg, phase)` so **placement is config, not code**. Each phase is independently reversible (`control_plane` ? `cloud_run_l4`) and falls back to in-process on a non-cloud box.

Phase	Change	Mechanism	Fallback	Status
P0	*Today* ? detect on L4; validate+stitch on CP	`compute_target=cloud_run`	in-process	shipped `[verified]`
P1	**Stitch ? L4**	`phase_placement.compile=cloud_run_l4` (+ `cloud_stitch_encoder`)	in-process stitch	**#521 (open)**
P2a	**Drop the CP re-hash** (trust submit-time md5)	`deployment.trust_submit_video_id` (default on)	re-hash on the drain	**this PR (win #2)**
P2b	**Validation ? L4** (CP stops running validators)	`phase_placement.validation=cloud_run_l4` (guards: requires cloud segment)	CP validates	**#521 (open)**
P2 net	**CP never downloads the file** ? the ~30 s gap dies; movement 3×?2×	P2a ? P2b together	either half alone degrades gracefully	needs #521 **+** win #2 merged
P3	Worker serializes its **validation verdict** back (per-validator results + #518 cache warm), so a worker-side reject renders the same SPA detail as a CP reject	new `report.json`/marker field + CP ingest	worker reject = rc?0 + message (today)	**follow-up** (see §6)
P4	*If and only if* sustained load crosses the §4.2 breakeven	persistent worker (VM/Service)	scale-to-zero Job	**owner-gated, new ADR**

The "collapse onto the L4" the epic asks for = P1 + P2, both config flips on #521's knob + this PR's win #2. No rearchitecture, no new infra, fully reversible.

6. Honest limits ? what this does NOT do / does NOT prove

- **A green suite does not prove the cloud path** ? the tests are fully-mocked/offline (a fake `CloudRunJobClient`). P2's "CP never downloads" and the idle-GPU/round-trip numbers are validated **in the cost model + code paths**, not on live infra. A live CUJ on rev ? `00020` (owner-side, D17 budget) is required to *confirm* the gap is gone and the movement is 2×.
- **Network-egress "free intra-region" is `[researched]`, not live-verified** ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were *not* free, A? is *still* cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.
- **Win #2 changes nothing when no trusted id is present** ? local/appliance uploads and non-GCS direct-URI submits (`submit_time_video_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).

- **P3 (worker validation-verdict serialization) is a real gap #521 leaves open** `[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-validator `ValidationResult` list (`result['results']` the SPA renders) and the #518 `validation_cache` warm are **not** reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.
- **SQLite is the residual CP scaling limit** ? the thin router is stateless *except* the SQLite DB; true horizontal scale-out needs Postgres (out of scope; noted).
- **The #513 asymmetry reverses under P2b** `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards `validation.*` as `--set`. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.

7. Deliverables & progress tracker ? source of truth

Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, branched from `origin/master`, no stacking.

ID	Deliverable	Depends on	Gated?	Status	PR
P0	**This design doc** (resolves topology A/B/C; recommends A?; specs durability + phased migration)	?	owner-review	? (this PR)	#524
P1	**Win #2** ? drop the CP re-hash; trust submit-time md5; `deployment.trust_submit_video_id` (default on), reversible	?	No	? (this PR)	#524
P2	**Win #1** ? validation-on-L4 (`phase_placement.validation`)	#521	No	? **delivered by #521** ? not duplicated here	#521
P3	**CP-download-avoidance measured live** ? confirm the ~30 s gap is gone + movement is 2× on rev ? `00020`	#521 + P1 merged	? owner-side (live spend, D17)	?	?
P4	**Worker validation-verdict serialization** (§6 gap) so P2b keeps rich SPA verdict + #518 cache	#521	No	? follow-up	?
P5	**Persistent-GPU re-evaluation** only if load crosses §4.2 breakeven	volume	? new ADR	?	?

8. Open questions ? owner / external

Blocking (owner sign-off):

1. **Do you accept rejecting "upload directly onto the L4" (L5)?** The epic proposed it; this doc recommends against it with the §4.2/§4.3 idle-GPU-vs-free-LAN-pull numbers. A? delivers the same "collapse onto the L4" outcome, cheaper and more durable, via config. *If you still want upload-onto-L4* (e.g. for a future no-bucket edge deployment), §4.5 specs its durability model ? but it needs a Service-with-GPU or VM (a D16(a)/D7 reversal ? new ADR).
2. **Flip P2b (validation?L4) before or after P3 (verdict serialization)?** P2b alone offloads CPU but degrades the reject-UX + skips #518 cache-warm (§6). Recommend: land P3 first, then P2b.

Nice-to-know:

3. Is pilot volume expected to cross the §4.2 breakeven within the planning horizon? (Decides whether P5 is ever live.)

9. Definition of done

- [x] Design doc resolves the topology question (A/B/C scored) with a clear recommendation (A?) + quantified cost/latency/reliability. `[this PR]`
- [x] Minimal control-plane surface defined (§4.4). `[this PR]`
- [x] Async copy-to-bucket durability model specified with failure/resume + exposure window (§4.5). `[this PR]`
- [x] Phased migration laid out on #521's `phase_placement`, each step a config flip with a bucket-mediated fallback (§5). `[this PR]`
- [x] Win #2 landed (config-guarded, reversible, tests green). `[this PR]`
- [ ] Win #1 confirmed delivered by #521 (not duplicated). `[cross-PR]`
- [ ] Live CUJ confirms the gap is gone + movement is 2× (P3, owner-side). `[gated]`
- [ ] Owner signs off on L5 (reject upload-onto-L4) ? flip Status ? IN PROGRESS/DONE.

10. References

```
218     **Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5),
`backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) /
`_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the
md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`),
`backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521
`resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-
validation`).
219     **Internal docs:** [`PHASE_PLACEMENT.md` (#521)](https://github.com/Khelsutra/badminton-
highlight-indexer/blob/feat/521-compile-
on-l4/docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md), [README.md](./README.md) (ADR-014,
D4/D7/D16), `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md` §9
(evidence).
220     **External `[verified]` rates:** GCP Cloud Billing Catalog API
(`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`:** same-
region GCS?Cloud Run egress treatment (not live-fetchable here).
221
222     ---
223
224     ### Changelog
225     - `2026-07-08` ? initial draft: topology A/B/C resolved (recommend A? = co-located Jobs
+ bucket upload; reject persistent-GPU + upload-onto-L4 with a cost model); durability + phased
migration on #521's `phase_placement`; win #2 landed alongside.
226
```

4. user (2026-07-08T06:36:28.735Z)

```
No files found
```

5. user (2026-07-08T06:36:34.254Z)

```
ls: cannot access 'runlogs/': No such file or directory
```

6. user (2026-07-08T06:36:38.645Z)

```
./docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md
---
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
```

7. user (2026-07-08T06:36:41.302Z)

```
1       # DEPLOY REPORT ? cloud-serving, validation-latency closure (#512 shared-decode + #513
worker skip-revalidation), 2026-07-07
2
3       > Durable **closure record** for the two validation-latency fixes now live on the
`cloud-serving` path: **#512** (`perf(validators)`: decode `scene_cut`/`blur`/`relevance` in ONE
shared pass instead of ~125 random per-sample seeks across 3 captures) and **#513**
(`fix(serving)`: the worker skips its redundant re-validation when the control-plane already
validated + dispatched, via `--skip-validation`). Both shipped in the live image `rally-
worker@sha256:8045f54e?` (built from `origin/master` @ `f5c9dd9`; #512 `20862d7`, #513 `23c1868`
are ancestors). Measures the before/after on 3 clips spread across codecs/resolutions, all
against the real serving infra. Project `gen-lang-client-0306026826`, region `asia-southeast1`.
Honest-limits (incl. the owner-gated live CUJ) in §6.
4
5       ## 1. TL;DR
6
7       | Stage | Result |
8       |---|---|
9       | Live image carries #512+#513 | ? control-plane rev `00018-qvz` + worker Job both on
`sha256:8045f54e?` (= `f5c9dd9`, ancestors `20862d7`/`23c1868` verified) |
```

Baseline captured (pre-fix)	? from prod logs on rev `00017-qbd` (image `09d2edf7` = `66756ad`, pre-#512/#513) ? no re-run needed
#513 worker skip-revalidation	? real dispatch-style exec: Video 1 validation **SKIPPED (0 s)**, runs to **SUCCESS / 13 rallies** ? the blur-reject that failed the live CUJ is **gone**
#512 shared-decode speedup (real L4)	? Video 1 re-validation **169.3 s ? 69.7 s (2.43×)** on identical worker hardware, decision byte-identical (sharpness 11.9 < 20.0 both)
#512 speedup (off-box A/B, breadth)	? 1080p H.264 **1.95×**, 4K HEVC **3.10×**, 5.3K 10-bit HEVC **3.23×**; every validator decision unchanged OLD?NEW
Live control-plane "after" number	? **live-corroborated 2026-07-08** (§9): rev `00019` validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the 2?3 min band) + reel compiled end-to-end. Video-1's exact 412 s?~170 s A/B stays a projection (the live CUJ used a different, heavier clip).

Bottom line: the two fixes do exactly what they claim on live serving hardware. **#513** turns the soft 5.3K clip from a **hard FAIL** (worker re-validated at its default `min_blur=20`, rejected sharpness 11.9, `rc2`, no reel) into a **PASS with 13 rallies** by trusting the control-plane's gate. **#512** cuts the frame-sampling validators ~**2?3×** (2.43× measured on the L4; 1.95?3.23× off-box across codecs) with **zero decision drift** OLD?NEW. The only number still owner-pending is the live control-plane validation wall on rev 00018 ? its components are measured and it projects to ~2?3 min from the ~6m52s baseline.

2. Resources

- **Control-plane:** `rally-control-plane` Cloud Run **service**, rev **`rally-control-plane-00018-qvz`** (100% traffic), image `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:8045f54ea9fcbf?` (built from `f5c9dd9`), `DEPLOYMENT_PROFILE=cloud-serving`, CPU-only **2 vCPU**, edge-auth ON (`edge_auth_enabled=true`, `cf_team_domain=khelsutra.cloudflareaccess.com`).
- **Worker:** `rally-worker` Cloud Run **Job**, same image `sha256:8045f54e?`, 8 vCPU / 32 GiB / 1× NVIDIA **L4**, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`, `WASB_DECODE_BACKEND=nvdec_resident`, GCSFuse mount of `gs://khelsutra-rally-serving`.
- **Baseline (pre-fix) surface:** control-plane rev **`00017-qbd`** / worker image `sha256:09d2edf7?` (= `66756ad` = #507+#508+#509, both **pre-#512/#513**) ? the source of the prod baseline logs below (2026-07-07, before the 16:15 rev-00018 cutover).
- **Corpus:** Video 1 `gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4` (1.41 GiB, 5312×2988 yuv420p10le HEVC, 3019 frames, MD5 `8fe6b719603dd7a7b2b6ef543139a4a3`); `gs://khelsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4` (1080p H.264); `gs://khelsutra-rally-corpus/videos/golden/GX010128.mp4` (2.18 GiB, 4K HEVC, 45298 frames).

3. Exact commands

```bash
# --- Provenance: prove the live image carries both fixes ---
gcloud run services describe rally-control-plane --region asia-southeast1 \
  --format="value(status.latestReadyRevisionName,
spec.template.spec.containers[0].image)"   # 00018-qvz  ?@sha256:8045f54e?
git merge-base --is-ancestor 20862d7 f5c9dd9 && echo "#512 in live image"   # yes
git merge-base --is-ancestor 23c1868 f5c9dd9 && echo "#513 in live image"   # yes

# --- #513 proof: worker A/B on the LIVE image (Video 1 already staged in the serving bucket) ---
V1="gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4"
# Run A ? WITH --skip-validation (exactly what orchestration._maybe_dispatch_remote emits on dispatch):
gcloud run jobs execute rally-worker --region asia-southeast1 --async \
  --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-serving/_valclose/v1-skipval@@--segmenter=wasb_hybrid@@--skip-validation@@--set=deployment.compute_target=local_gpu@@--set=indexing.detector_impl=native@@--set=indexing.skip_ai_handoff=true@@--set=indexing.wasb.batch_size=64@@--set=indexing.wasb.prefetch_batches=4@@--set=indexing.wasb.timeout_sec=0"
# Run B ? WITHOUT --skip-validation (worker re-validates with the NEW #512 validators ? still blur-rejects):
gcloud run jobs execute rally-worker --region asia-southeast1 --async \
  --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-serving/_valclose/v
```

1-reval@@--segmenter=wasb_hybrid@@--set=deployment.compute_target=local_gpu@@--
set=indexing.detector_impl=native@@--set=indexing.skip_ai_handoff=true"
40
41   # --- #512 timing: bracket "pulled ? -> in.mp4" ? "validation FAILED/SKIPPED" ?
"Phase 2 (WASB) completed" ---
42   gcloud logging read 'resource.type="cloud_run_job" AND
labels."run.googleapis.com/execution_name"="rally-worker-<exec>"' \
43   --freshness 1d --format="value(timestamp, textPayload)"
44
45   # --- #512 off-box A/B (isolated clones; run_all decodes the SAME sample indices in both
trees) ---
46   git clone --local --no-hardlinks <repo> repo_old && git -C repo_old checkout 23c1868   #
pre-#512 (per-sample seeks)
47   git -C repo_old worktree add --detach repo_new f5c9dd9
live image src (shared pass)
48   python valbench.py <repo_old|repo_new> <video> <label> 2   # times
registry.run_all(video, load_config())
49   ```
50
51   ## 4. Run results
52
53   ### 4a. Video 1 (5312×2988 10-bit HEVC, 3019 frames) ? the clip that motivated #512/#513
54
55 | Metric | BEFORE ? pre-fix (rev `00017`, img `09d2edf7`) | AFTER ? #512+#513 live (rev
`00018`, img `8045f54e`) |
56 |---|---|---|
57 | Control-plane validation | **411?412 s** (6m52s) ? passed (baked `min_blur=8.0`) |
projected ~170 s (2?3 min); **live-corroborated 2026-07-08 ? 183 s** on rev `00019` (§9 ? a
fresh 1.83 GiB clip, not a Video-1 re-run) |
58 | Worker (re-)validation | **169.3 s** ? ? **BLUR-REJECT** (sharpness 11.9 < default
20.0) | **0 s ? SKIPPED** (`--skip-validation`, #513) |
59 | ? counterfactual: worker re-val if NOT skipped | *(169.3 s, OLD
validators)* | **69.7 s** (NEW #512 validators, **2.43×**, same 11.9<20 decision) |
60 | End-to-end | ~10m31s wall ? **FAILED** (0 rallies, no reel) | worker ? **SUCCESS** in
~2m38s exec ? **13 rallies**; **reel live-confirmed 2026-07-08** (§9 ? fresh-clip CUJ compiled
an 11-clip reel end-to-end) |
61 | Rallies produced | **0** (rejected before detection) | **13** (`report.json`
`status=success`, 13 segments) |
62 | Pass / fail | ? **FAIL** | ? **PASS** |
63
64   **Prod baseline evidence (rev 00017, 2026-07-07, unmodified):** control-plane `13:49:18
Running validations ? 13:56:10 Video passed checks` = 411.4 s (and `14:03:56 ? 14:10:48` = 411.9
s). Worker exec `rally-worker-2lbz2` (dispatched by the 14:03 run, OLD image): `pulled 14:11:35
? validation FAILED: blur_check ? sharpness 11.9 below 20.0 @14:14:24 ? exit(2)` ? a
successful GPU dispatch that failed after the fact, the exact CUJ #513 fixes.
65
66   **After evidence (live image `8045f54e`, 2026-07-07):**
67   - `rally-worker-k29rk` (**Run A, --skip-validation**): `pulled 17:09:24 ? [worker]
validation SKIPPED (--skip-validation) 17:09:27 ? native WASB (torch 2.6.0+cu124, cuda True) ?
Phase 2 (WASB) completed in 100.9 s, 13 candidate windows (1510/3019 visible) ? emitted 13
rallies ? exit(0)`. Exec wall 157.7 s; `gpu_active`?100.9 s (WASB);
`bytes_out`=`report.json`+`intervals.csv`. Output
`gs://?/_valclose/v1-skipval/outputs/8fe6b719?/`, `status=success`, 13 segments.
68   - `rally-worker-t8llx` (**Run B, re-validation on the NEW image**): `pulled 17:09:29 ?
validation FAILED: blur_check ? sharpness 11.9 below 20.0 @17:10:39 ? exit(2)`. Re-val wall
69.7 s vs the OLD image's 169.3 s = **2.43× on identical L4 hardware**, and the **decision
is byte-identical** (11.9 < 20.0) ? #512 is pure speed, zero behavior change. (This run also
shows *why* #513 is needed: absent the skip, the worker's default `min_blur=20` still rejects
the clip the control-plane passed at 8.0.)
69
70   ### 4b. Off-box #512 A/B ? codec/resolution breadth (16-core local box, cv2 4.13.0,
warm, `run_all`)
71
72   Isolated clones at `23c1868` (OLD, per-sample seeks) vs `f5c9dd9` (NEW, shared pass);
identical sample indices ? a fair A/B. **Every validator decision is identical OLD?NEW** on

every clip (no PASS/FAIL drift from the fix).

Clip	codec / res	OLD (warm)	NEW (warm)	Speedup	decisions OLD?NEW
Boxhill_Doubles	1080p H.264	10.91 s	5.60 s	**1.95×**	6/6 identical (all PASS)
GX010128	4K HEVC (45298 fr)	35.76 s	11.54 s	**3.10×**	6/6 identical (all PASS)
Video 1	5.3K 10-bit HEVC	173.20 s	53.62 s	**3.23×**	6/6 identical (all PASS?)

? On this Windows/cv2 box the 10-bit-HEVC decode yields a Laplacian variance **above** 20 so blur PASSES off-box, whereas the Linux serving stack measures **11.9** and rejects. That platform decode-stack difference is **orthogonal to #512** (which changes only *which frames are sampled*, identically in both trees); the authoritative blur verdict and the real speedup come from the L4/prod (§4a). Off-box contributes only the OLD?NEW **ratio**, which is decode-stack-independent.

The win grows with per-frame decode cost (random-seek re-decode is what #512 removes), and it **triangulates** with the real-L4 Video 1 A/B (2.43×): the off-box ratios bound the control-plane projection.

4c. Control-plane projection (2-vCPU)

Baseline **~412 s**. Applying the L4-measured **2.43×** ? **~170 s (~2m50s)**; the off-box ratios (1.95?3.23×) bracket **~2?3 min**. Matches the pre-fix design estimate (~6.5 min ? ~2 min). Exact live number is owner-gated (§6).

5. Rollout state

- **Already deployed ? this is a measurement/closure record, not a deploy.** #512+#513 went live with the `8045f54e` image cutover (control-plane rev `00018-qvz`, worker Job same digest) on 2026-07-07. No infra change was made for this report.
- **Rollback (pre-existing levers):** control-plane `gcloud run services update-traffic rally-control-plane --region asia-southeast1 --to-revisions rally-control-plane-00017-qbd=100`; worker `gcloud run jobs update rally-worker --region asia-southeast1 --image ?/rally-worker@sha256:09d2edf7?`. Both revert to the pre-#512/#513 image.
- **Scratch outputs** written under `gs://khelsutra-rally-serving/_valclose/` (isolated prefix; safe to delete).

6. Honest limits / not-verified

- **Live end-to-end CUJ is owner-gated.** ? **CLOSED 2026-07-08 ? see §9** (owner drove one live SPA upload on rev `00019`; both ? cells now measured). Original limit, retained for context: `POST /api/process` on rev 00018 requires a Cloudflare-Access JWT (`edge_auth_enabled=true`; a direct call returns **401** `missing Cf-Access-Jwt-Assertion header`). So the two ? cells ? the **live control-plane validation wall on rev 00018** and the **stitched reel** ? need the owner to drive one upload (or hand over a CF-Access JWT / service token). Everything else here is measured on the real serving infra. **Recipe to close them:** upload "Video 1 - Badminton.MP4" via the SPA (compute=cloud), then `gcloud logging read '?service_name="rally-control-plane"?("Running validations" OR "passed checks")'` for the wall, and confirm the dispatched worker exec logs `validation SKIPPED` + emits rallies + the reel compiles. Expect CP validation ~2?3 min and a PASS.
- **#513's worker win is shown via a dispatch-faithful direct exec**, not a control-plane-initiated dispatch (same auth gate). The `--skip-validation` flag, its emission in `orchestration._maybe_dispatch_remote`, and its handling in `worker.cmd_infer` are unit-tested on master; Run A exercises the exact worker code path with the exact flag. ? **UPDATE 2026-07-08 (§9): now also proven on a genuine control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted `--skip-validation` on a live SPA run and the worker honored it (`validation SKIPPED`, 0 re-validation).
- **Rallies, not a reel.** Run A used `skip_ai_handoff=true` (secret-free) so the 13 "rallies" are local candidate windows (no Gemini refine, no stitched mp4) ? this validates decode+detect+windowing end-to-end and that the clip **passes**; the reel stitch is the owner-CUJ step above. The 13 segments are its inputs.
- **Off-box absolute times are from a Windows 16-core box (cv2/FFmpeg), not the 2-vCPU Linux control-plane** ? so the **ratios** transfer (the change is algorithmic: seeks?shared forward-hybrid pass), the absolutes don't, and (per ?) the 10-bit blur *verdict* can differ; the control-plane absolute is anchored to the prod 412 s baseline, not the local numbers. Video 1

off-box is a cross-check of the authoritative real-L4 2.43×.
97   - **#515** (crash-before-marker fast-fail) is NOT in this image (`ef41473` post-dates `f5c9dd9`); it's unrelated to validation latency and out of scope for this closure.
98
99   ## 7. Cost
100   - 2 L4 worker execs (Run A 157.7 s + Run B 109.6 s ? 268 s of L4 time) ? **~$0.5?0.7**. Off-box A/B: **$0** (local). No VM, no image build. GCS read for the 3 downloaded clips (~5.1 GiB) is intra-project.
101
102   ## 8. Closure verdict
103   ? **CLOSED ? #512 and #513 are proven on live cloud-serving hardware.** #513 converts the motivating soft-focus 5.3K clip from a post-dispatch FAIL into a 13-rally PASS by eliminating the worker's redundant re-validation (169 s + wrongful blur-reject ? 0 s); #512 speeds the frame-sampling validators **2.43× on the real L4** and **1.95?3.23× off-box across codecs**, with **zero decision drift** OLD?NEW on every clip. The last owner-gated residual is now **closed live 2026-07-08 (§9)**: an owner-driven SPA CUJ on rev `00019` validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the projected 2?3 min band) and compiled an 11-clip reel end-to-end ? with the worker `validation SKIPPED` on a genuine control-plane dispatch. (The Video-1-specific 412 s?~170 s A/B stays a projection by design ? the live CUJ used a different, heavier clip.) Validation-latency work is done.
104
105   ## 9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)
106
107   The two ? cells in §4a (the **live control-plane validation wall** and the **end-to-end reel**) were owner-gated on the Cloudflare-Access `POST /api/process`. On **2026-07-08** the owner drove one real SPA upload (compute=**Cloud**) end-to-end, so both are now measured on live serving infra. Mirrors the gpu-resident report's "§5 Update" append ? the body above is unchanged; this section records what the live run added.
108
109   **Two honest deltas from the recipe** (state moved after this report's 2026-07-07 image cutover):
110   1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **`rally-control-plane-00019-96f`**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed 2026-07-07T17:06Z ? ~51 min after the `00018`/`8045f54e` cutover §2 documents; the worker Job is on the same `:latest`=`c6393d27` digest (both surfaces match). It's an undocumented forward-bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the `00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
111   2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"** ? **1.83 GiB** (1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? *heavier* than Video 1's 1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different clip has a different absolute wall).
112
113   **Full phase breakdown (UTC 2026-07-08, rev `00019`):**
114
115 | # | Phase | Window | Wall | Machine | Notes |
116 |---|---|---|---|---|---|
117 | 1 | Upload (client?server, 80-part multipart) | 04:25:56 ? 04:28:51 | **~2m55s** | client uplink | 1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra |
118 | 2 | Accept ? validation start (stage) | 04:28:52 ? 04:29:22 | ~30 s | control-plane | stage upload to serving bucket |
119 | 3 | **Validation (#512 shared-decode)** ? | 04:29:22 ? 04:32:25 | **183 s (3m03s)** | control-plane 2 vCPU | one-pass scene_cut/blur/relevance; PASSED |
120 | 4 | Dispatch ? worker ready (cold start) | 04:32:25 ? 04:33:20 | ~55 s | Cloud Run Job | image pull + GCSFuse mount + torch/cuda init |
121 | 5 | **Worker skip-validation (#513)** ? | 04:33:20 ? 04:33:24 | ~4 s | L4 | `validation SKIPPED`, 0 re-validation |
122 | 6 | Detection / segment (WASB) | 04:33:24 ? 04:36:20 | **161.6 s** | L4 (cuda) | 15 candidate windows, 2561 visible pts |
123 | 7 | Ingest + finalize | 04:36:20 ? 04:36:24 | ~4 s | control-plane | intervals?DB, `status=success` |
124 | ? | (user reviews rallies, clicks Compile) | 04:36:24 ? 04:37:26 | ~62 s idle | ? | human-in-the-loop |

```
125     | 8 | **Compile / stitch** ? slow | 04:37:26 ? 04:50:21 | **~12m55s** | control-plane 2
vCPU | 11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s |
126
127     Server pipeline (process?success, excl. upload+idle): **7m32s**. Full CUJ (upload?reel):
**~24m25s** wall.
128
129     **Cell 1 ? control-plane validation AFTER (live):** **183 s (3m03s)** on rev `00019`,
PASSED ? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected
**2?3 min** band on a clip *heavier* than Video 1 (pre-#512 per-sample seeks were the
~6m52s-class baseline). Evidence: `Running validations 04:29:22.789 ? Video passed checks
04:32:25.780`.
130
131     **Cell 2 ? end-to-end reel (live):** `/api/compile` stitched **11 of the 15** detected
rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khelsutra-
rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved
04:50:10 *"Dynamic highlight reel saved (581.8 MB)"*, mirrored 04:50:21, served via
`/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch
? per-clip trim+watermark ? concat ? mirror-upload, all exercised.
132
133     **Bonus ? #513 proof upgraded** from the §6 dispatch-faithful *direct exec* to a genuine
**control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted
`--skip-validation` on a live SPA run and the worker honored it (0 re-validation).
134
135     **? Finding ? stitch is the scaling bottleneck (follow-ups filed).** The compile ran
**~13 min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a
full 4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized
by `_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't
scale past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make
phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a
low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).
136
137     **Cost (heuristic):** one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55**
(rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ?
**~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **<
$0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but
the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.
138
139     **Net:** both owner-gated cells are now live-measured on rev `00019` ? validation wall
**183 s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass
validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The
Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).
140
```

8. assistant (2026-07-08T06:37:02.849Z)

Now let me examine the actual code to determine what genuinely happens in the ~30s "Accept?validate" gap.

9. user (2026-07-08T06:37:04.320Z)

```
700         }
701         logger.info(
702             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
703             result.execution,
704             provenance["job"],
705             provenance["region"],
706             result.outputs_uri,
707             video_id,
708         )
709         return (
710             {
711                 "video_id": video_id,
712                 "status": "success",
713                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
```

```
'{seg_name}').",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
                "compute": provenance,
            },
            True,
        )


    def _execute_processing(
        config=None, db=None, req=None, progress=None, trusted_video_id=None
    ) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
        module-level ``global_config``/``db_instance`` (backward-compatible with existing
        tests that pass only ``req``). Route handlers always pass explicit params; older
        test code that calls ``_execute_processing(req)`` still works.

        ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.

        ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows
for this
        job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
        ``compute_video_id`` and already trusted for the submit-time cache probe. When
present (and
        ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-
hashing the
        multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes
as before.
        It is a keyword-only, internal channel ? NOT a client-supplied request field (a
caller's
        ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative
id).

        Raises on unexpected internal errors ? the caller records those as a job failure
        """
        import backend.main as _main  # call-time seam: main's globals/model stay patchable

        # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
        # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
        if config is not None and not isinstance(config, AppConfig):
            # Old signature: _execute_processing(req, progress=None)
            if req is None and db is not None and isinstance(db, _main.ProcessRequest):
                req = db
                db = None
            elif req is None and isinstance(config, _main.ProcessRequest):
                req = config
                config = None
        # Resolve config/db from module globals when not passed explicitly
        if config is None:
            config = _main.global_config
        if db is None:
            db = _main.db_instance

        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
```

```
reading
768         # consumers (hash / validators / segmenter) use the resolved local path.
769         local_video = ""
770         try:
771             local_video = storage.acquire_local_input(
772                 req.video_path, getattr(config, "storage", None)
773             )
774             # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
the whole
775             # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
776             # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
submit-time
777             # cache probe trusts, so reusing it here is consistent, not a new trust
assumption. Gated by
778             # deployment.trust_submit_video_id (default on) and only honoured when the id is
well-formed
779             # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
780             # is unchanged; only the redundant hash is skipped.
781             trust_submitted = bool(
782                 getattr(
783                     getattr(config, "deployment", None), "trust_submit_video_id", True
784                 )
785             )
786             # STRICT: trust only an id in the EXACT form the drain would have produced
(32-char
787             # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
? a
788             # differently-cased or malformed id is a signal something is off, so re-hash
rather than
789             # silently coerce it into the DB key.
790             candidate = trusted_video_id or ""
791             if trust_submitted and _looks_like_video_id(candidate):
792                 video_id = candidate
793                 logger.info(
794                     "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
795                     video_id,
796                 )
797             else:
798                 video_id = _main.compute_video_id(local_video)
799         except Exception:
```

10. user (2026-07-08T06:37:10.214Z)

```
1       """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3       Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4       proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2). Where they assemble depends
5       on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7       * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
8         file under security.upload_dir, exactly as before.
9       * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10        through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11        while an incremental MD5 accumulates ? the assembled file never touches local
12        disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
13        (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
14        returns the staged URI (inside the #465 input jail by construction ? it is under
15        ``input_root``) plus the streamed md5 as ``video_id``.
16
17      Upload state is in-process by design (this is the single-box alpha): a server restart
18      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
19      arrives with PR-3 identity scoping.
20
```

```
    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
    reads the live value the server set at startup (and so tests that monkeypatch
    ``backend.main.global_config`` flow through unchanged).
    """

    import hashlib
    import os
    import threading
    import unicodedata
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel
    from starlette.concurrency import run_in_threadpool

    from backend import storage
    from backend.utils.app_home import (
        resolve_output_dir,  # P0a: app-home-aware path resolution
    )
    from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main

        sec = getattr(_main.global_config, "security", None)
        upload_dir = resolve_output_dir(
            getattr(sec, "upload_dir", None) or "output/uploads"
        )  # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def upload_caps(cfg) -> Dict[str, int]:
        """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).

        ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
?
        without it the guard is dead code and the user learns from a mid-upload 413.
```

```
        ``max_part_mb`` is informational: each /api/upload/init response carries the
        authoritative value the client actually chunks by."""
        sec = getattr(cfg, "security", None)
        return {
            "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
            "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
        }


    def _staging_target() -> "Optional[Dict[str, Any]]":
        """The remote staging root + storage settings, or None for the local-file flow.

        Remote staging is selected by the same input config /api/process resolves against
        (storage.input_backend + input_root), so a staged upload is submittable verbatim.
        """
        import backend.main as _main

        sc = getattr(_main.global_config, "storage", None)
        if sc is None:
            return None
        input_backend = getattr(sc, "input_backend", "local") or "local"
        input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
        if input_backend == "local" or not input_root:
            return None
        cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
        return {"root": input_root, "config": cfg}


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if not st:
            return
        writer = st.get("writer")
        if writer is not None:
            # A streaming writer has no cancel ? close() finalizes whatever was staged, then
            # the partial object is deleted. Both are best-effort: the upload is already
            # aborted from the client's point of view.
            try:
                writer.close()
            except Exception:  # noqa: BLE001 ? abort cleanup must never mask the abort
                pass
            try:
                ref = storage.parse_uri(st["stage_uri"])
                storage.get_storage(ref.backend, config=st.get("config")).delete(ref)
            except Exception:  # noqa: BLE001
                pass
            return
        try:
            os.remove(st["tmp_path"])
        except OSError:
            pass


    def _normalize_upload_filename(filename: str) -> str:
        """Normalize browser-provided file names before extension validation.

        Some mobile/file-provider pickers can hand us visually normal names with
compatibility
        characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
existing
        traversal/null-byte checks in ``safe_output_filename`` as the authority after this
cleanup.
        """
        return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
```

```


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(_normalize_upload_filename(req.filename))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(
                status_code=400,
                detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
            )
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(
                status_code=413,
                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
            )
        upload_id = uuid.uuid4().hex
        staging = _staging_target()
        if staging is not None:
            # Remote staging: stream parts straight into the bucket under the input root ?
            # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
            # upload_id in the key keeps concurrent same-name uploads distinct, and the
free-
            # space guard is skipped: no local bytes are consumed.
            stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
            try:
                writer = storage.open_writer(stage_uri, config=staging["config"])
            except NotImplementedError:
                staging = None  # backend can't stream (e.g. mounted-Drive) ? stage locally
            else:
                with _uploads_lock:
                    _uploads[upload_id] = {
                        "filename": name,
                        "writer": writer,
                        "hasher": hashlib.md5(),
                        "stage_uri": stage_uri,
                        "config": staging["config"],
                        "parts": 0,
                        "bytes": 0,
                    }
                return {
                    "upload_id": upload_id,
                    "max_part_mb": part_bytes // (1024 * 1024),
                    "next_part": f"/api/upload/{upload_id}/part/0",
                }
        os.makedirs(upload_dir, exist_ok=True)
        # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
        # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
        # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
        require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {
                "filename": name,
                "tmp_path": tmp_path,
                "parts": 0,
```

```
                "bytes": 0,
            }
        return {
            "upload_id": upload_id,
            "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0",
        }


    @router.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(
                status_code=409,
                detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
            )

        received = 0
        overflow = None
        if st.get("writer") is not None:
            # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
an
            # oversize stream), then hand it to the blocking bucket writer in one threadpool
            # hop so a network flush never stalls the event loop mid-upload.
            buf = bytearray()
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = (
                        f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                    )
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = (
                        f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                    )
                    break
                buf += chunk
            if not overflow and buf:
                data = bytes(buf)
                await run_in_threadpool(st["writer"].write, data)
                st["hasher"].update(data)
        else:
            with open(st["tmp_path"], "ab") as fh:
                async for chunk in request.stream():
                    received += len(chunk)
                    if received > part_bytes:
                        overflow = (
                            f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
                        )
                        break
                    if st["bytes"] + received > max_bytes:
                        overflow = (
                            f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
                        )
                        break
```

```
                    fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(
                status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
            )

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {
            "upload_id": upload_id,
            "received_parts": st["parts"],
            "received_bytes": st["bytes"],
        }


    @router.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(
                status_code=409,
                detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
            )
        if st.get("writer") is not None:
            # Remote staging: close() finalizes the streamed object; the md5 accumulated per
            # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
            # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
            # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
            try:
                st["writer"].close()
            except Exception as e:  # noqa: BLE001 ? surface a clean verdict, then clean up
                _abort_upload(upload_id)
                raise HTTPException(
                    status_code=502,
                    detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
                ) from e
            with _uploads_lock:
                _uploads.pop(upload_id, None)
            return {
                "path": st["stage_uri"],
                "video_id": st["hasher"].hexdigest(),
                "size_bytes": st["bytes"],
                "next": "POST /api/process with this path",
            }
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(
            final_path
        ):  # never overwrite an earlier upload with the same name
            stem, ext = os.path.splitext(st["filename"])
            final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
        os.replace(st["tmp_path"], final_path)
        with _uploads_lock:
            _uploads.pop(upload_id, None)
        video_id = compute_video_id(final_path)
        return {
```

```
331             "path": os.path.abspath(final_path),
332             "video_id": video_id,
333             "size_bytes": os.path.getsize(final_path),
334             "next": "POST /api/process with this path",
335         }
336
337
338     @router.delete("/api/upload/{upload_id}")
339     def upload_abort(upload_id: str):
340         """Abort an in-flight upload and delete its partial file."""
341         with _uploads_lock:
342             known = upload_id in _uploads
343         if not known:
344             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345         _abort_upload(upload_id)
346         return {"status": "aborted", "upload_id": upload_id}
347
```

11. assistant (2026-07-08T06:38:18.363Z)

```
Let me confirm `acquire_local_input` genuinely downloads a GCS URI to scratch (central to
whether the doc's causal claim, or the runlog's label, is the accurate one).
```

12. user (2026-07-08T06:38:20.643Z)

```
Exit code 2
backend/storage/__init__.py:77:def acquire_local_input(
```

13. user (2026-07-08T06:38:23.517Z)

```
77      def acquire_local_input(
78          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
79      ) -> str:
80          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
81
82          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
83          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
84          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
85          path returned. The caller owns successful remote scratch copies; use
86          ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
87          sc = _storage_dict(storage_config)
88          ref = resolve_input_ref(raw, sc)
89          if ref.backend == "local":
90              return (
91                  ref.key
92              )  # identity ? no temp, no copy, byte-identical to passing the path directly
93          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
94          try:
95              return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
96          except Exception:
97              shutil.rmtree(scratch, ignore_errors=True)
98              raise
99
100
101     def cleanup_acquired_local_input(
102         raw: str, local_path: str, storage_config: Any = None
103     ) -> None:
104         """Delete a remote scratch dir produced by ``acquire_local_input``.
105
```

```
106         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
107         into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
108         finished hashing/validating/stitching/segmenting the returned file.
109         """
110         try:
111             if input_is_local(raw, storage_config):
112                 return
113         except ValueError:
114             return
115         if not local_path:
116             return
117         scratch = os.path.dirname(os.path.abspath(local_path))
118         if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
119             shutil.rmtree(scratch, ignore_errors=True)
120
121
122     def _backend_class(name: str):
123         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
124         if name == "local":
125             from backend.storage.local import LocalStorage
126
127             return LocalStorage
128         if name == "s3":
129             from backend.storage.s3 import S3Storage
130
131             return S3Storage
132         if name == "gcs":
133             from backend.storage.gcs import GCSStorage
134
135             return GCSStorage
136         if name == "gdrive":
137             from backend.storage.gdrive import GDriveStorage
138
139             return GDriveStorage
140         if name == "gdrive-api":
141             from backend.storage.gdrive_api import GDriveApiStorage
142
143             return GDriveApiStorage
144         raise ValueError(
145             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146         )
```

14. assistant (2026-07-08T06:38:54.559Z)

I have verified the finding against the actual code and documents. Here is my assessment.

What the doc claims (L4_PRIMARY_TOPOLOGY.md):
- §4.1 line 68: `Accept ? validate start | ~30 s | control-plane | ... | **the gap** = CP re-download to scratch (acquire_local_input + re-hash)`, under a §4.1 header tagged `[verified]` (line 61) that reads "From runlog §9" (line 63).
- §1 line 23: the `~30 s pre-validation gap` is "unlogged" and equals finding (b) = CP re-downloads to scratch (`acquire_local_input`) + redundant re-hash (`compute_video_id`).

What the runlog says (runlog §9 line 118): the same phase "Accept ? validation start (stage)" ~30 s is labeled "stage upload to serving bucket".

What the code actually does:
- `backend/api/uploads.py:169-195` ? a GCS/remote-input upload STREAMS parts straight into `<input_root>/uploads/<upload_id>/<name>` in the serving bucket. There is no separate bucket-to-bucket "stage" step; the durable serving-bucket copy IS the upload. So the runlog's "stage upload to serving bucket" describes work that does not occur in that window.
- `backend/orchestration.py:771` ? `_execute_processing` calls `storage.acquire_local_input(req.video_path, ...)`, and `backend/storage/__init__.py:88-95`

confirms a `gs://` URI is `fetch`ed (downloaded) into a fresh scratch dir. Then `backend/orchestration.py:798` re-hashes via `compute_video_id`. So the doc's causal attribution (download-to-scratch + re-hash) is the code-accurate one.

Verdict: REFUTED. The doc's causal claim is factually correct and verified against the code it cites; the runlog's label is the imprecise one (the finding itself concedes this). Three reasons it is not a defect a maintainer must fix:
1. "unlogged" fairly reads as "no application-log marker for the CP's own download" ? it appears in explicit contrast to the worker's logged `[worker] pulled ?` on the same line 23. The runlog's row-2 "label" is not an app-log line but the runlog author's post-hoc interpretation of a timestamp gap (04:28:51?04:29:22), so "unlogged" is not contradicted by a runlog table row existing.
2. §4.1's `[verified]` tag covers the measured pipeline (phase/wall/machine columns), which match runlog §9 exactly; the Notes column's cause is verified against the cited code (`orchestration.py` `acquire_local_input` + `compute_video_id`, also anchored in §1 and §10), and the doc's honesty taxonomy (line 7) defines `[verified]` as "checked against code/runlog", satisfied here by code.
3. The proposed fixes are not cleanly actionable: retagging a code-verified claim down to `[researched]`/`[design]` would make the doc LESS accurate (the claim IS verifiable against code), and "correct the runlog phase label" edits a closed historical runlog this PR does not touch. At most this warrants an optional one-line footnote reconciling the runlog's imprecise "stage" label ? a clarity nicety, not a must-fix defect.

Files checked: `docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md:15,23,61-68`; `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md:117-119`; `backend/api/uploads.py:169-195,299-319`; `backend/orchestration.py:771,798`; `backend/storage/__init__.py:77-98`.

15. user (2026-07-08T06:39:06.796Z)

Structured output provided successfully